MEMOIZATION vs TABULATION

Java Dynamic Programming • Deep Concepts • IntelliJ Project • Full Runnable Code

A focused guide to both DP approaches: how they work, how to convert one into the other, when to choose each, and how to explain them in interviews.

Java | IntelliJ IDEA | Interview preparation

1. DP foundation: why these two approaches exist

Dynamic Programming (DP) is a technique for solving problems with overlapping subproblems and optimal substructure. Instead of recomputing the same smaller problem repeatedly, DP stores its answer and reuses it.

The two approaches

Memoization (Top-Down DP): start from the original question, solve it recursively, and store each computed state in a cache. A state is calculated only when it is first needed.

Tabulation (Bottom-Up DP): start from the smallest/base states and fill a table in an order that guarantees every dependency is already computed.

Terminology correction

The DP term is memoization, not memorization. Memorization means remembering information; memoization means caching function results so the same state is not solved repeatedly.

What must be true for DP?

• Overlapping subproblems: the recursion reaches the same state more than once.

• Optimal substructure: the answer to a state can be built from answers to smaller states.

• State definition must be precise: for example, dp[i] = number of ways to reach step i.

• Transition must list all legal previous states/choices.

• Base cases must match the smallest valid inputs.

2. Memoization — top-down approach

How it executes

• Call the recursive function for the original state.

• Check base cases first.

• Check whether the state is already present in the memo array/map.

• If cached, return immediately.

• Otherwise solve smaller states recursively.

• Store the computed answer before returning it.

Generic Java template

```
static int solve(int state, int[] memo) {
    if (/* base case */) return /* base answer */;
    if (memo[state] != UNCOMPUTED) return memo[state];

    int answer = /* combine solve(smallerState, memo) */;
    memo[state] = answer;
    return answer;
}
```

Use a sentinel that cannot be a valid answer, or use a separate boolean[] visited. If zero is a valid answer, initializing memo with zero and treating zero as “not computed” is incorrect.

Memoization flow

```
solve(n)
  |
  +-- Is n a base state? -------- yes --> return base answer
  |
  no
  |
  +-- Is memo[n] already computed? -- yes --> return memo[n]
  |
  no
  |
  +-- Recursively solve required smaller states
  |
  +-- Combine their answers
```

```
|
+-- Store memo[n] and return it
```

3. Fibonacci using memoization

File: src/dp/memoization/FibonacciMemo.java

Concept / state definition: fib(n) means the nth Fibonacci number. Each state depends on n-1 and n-2. The memo array prevents repeated work.

```
package dp.memoization;

import java.util.Arrays;

public class FibonacciMemo {
    static long fib(int n, long[] memo) {
        if (n <= 1) return n;

        if (memo[n] != -1) {
            return memo[n];
        }

        long answer = fib(n - 1, memo) + fib(n - 2, memo);
        memo[n] = answer;
        return answer;
    }

    public static void main(String[] args) {
        int n = 10;
        long[] memo = new long[n + 1];
        Arrays.fill(memo, -1);

        System.out.println(fib(n, memo)); // 55
    }
}
```

Complexity: Time O(n), cache space O(n), recursion stack O(n).

4. Climbing Stairs using memoization

File: src/dp/memoization/ClimbingStairsMemo.java

Concept / state definition: ways(n) means the number of ways to reach stair n. The last jump is either 1 or 2 steps, so ways(n)=ways(n-1)+ways(n-2).

```
package dp.memoization;

import java.util.Arrays;

public class ClimbingStairsMemo {
    static int ways(int n, int[] memo) {
        if (n == 0 || n == 1) return 1;
        if (memo[n] != -1) return memo[n];

        memo[n] = ways(n - 1, memo) + ways(n - 2, memo);
        return memo[n];
    }

    public static void main(String[] args) {
        int n = 5;
        int[] memo = new int[n + 1];
        Arrays.fill(memo, -1);
        System.out.println(ways(n, memo)); // 8
    }
}
```

Complexity: Time O(n), cache space O(n), recursion stack O(n).

5. Minimum Coin Change using memoization

File: src/dp/memoization/MinCoinsMemo.java

Concept / state definition: solve(amount) means minimum number of coins needed to make amount. Try every coin that does not exceed the remaining amount. -1 marks an impossible amount in the memo; INF marks an uncomputed state.

```
package dp.memoization;

import java.util.Arrays;

public class MinCoinsMemo {
    static int solve(int amount, int[] coins, int[] memo) {
        if (amount == 0) return 0;
```

```
        if (amount < 0) return Integer.MAX_VALUE / 2;

        if (memo[amount] != -1) return memo[amount];

        int best = Integer.MAX_VALUE / 2;
        for (int coin : coins) {
            int sub = solve(amount - coin, coins, memo);
            best = Math.min(best, sub + 1);
        }
        memo[amount] = best;
        return best;
    }

    public static void main(String[] args) {
        int[] coins = {1, 2, 5};
        int amount = 11;
        int[] memo = new int[amount + 1];
        Arrays.fill(memo, -1);

        int answer = solve(amount, coins, memo);
        System.out.println(answer >= Integer.MAX_VALUE / 2 ? -1 : answer); // 3
    }
}
```

Complexity: Time O(A·C), memo space O(A), recursion stack up to O(A); A=amount, C=number of coin types. Assumes positive coin values.

6. Tabulation — bottom-up approach

How it executes

• Define what every dp cell means.

• Initialize base states directly.

• Choose a loop order so dependencies are ready before a state is calculated.

• Fill each state iteratively using the transition.

• Return the cell that represents the original question.

Generic Java template

```
int[] dp = new int[n + 1];

// 1. Base case
dp[0] = BASE_ANSWER;

// 2. Build from small states to large states
for (int state = 1; state <= n; state++) {
    dp[state] = /* combine already-computed states */;
}

// 3. Read the required state
return dp[n];
```

Tabulation does not mean “always loop from 0 to n”. The correct order depends on dependencies. For interval DP, short intervals come before long intervals; for DAG DP, process nodes in topological order.

7. Fibonacci using tabulation

File: src/dp/tabulation/FibonacciTabulation.java

Concept / state definition: dp[i] means the ith Fibonacci number. Since dp[i] depends on dp[i-1] and dp[i-2], calculate i in increasing order.

```
package dp.tabulation;

public class FibonacciTabulation {
    static long fib(int n) {
        if (n <= 1) return n;

        long[] dp = new long[n + 1];
        dp[0] = 0;
        dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }
```

```
    public static void main(String[] args) {
        System.out.println(fib(10)); // 55
    }
}
```

Complexity: Time O(n), space O(n); can be reduced to O(1) because only two previous states are needed.

8. Climbing Stairs using tabulation

File: src/dp/tabulation/ClimbingStairsTabulation.java

Concept / state definition: dp[i] means ways to reach stair i. Base: dp[0]=1 and dp[1]=1. Transition: dp[i]=dp[i-1]+dp[i-2].

```
package dp.tabulation;

public class ClimbingStairsTabulation {
    static int ways(int n) {
        if (n == 0) return 1;

        int[] dp = new int[n + 1];
        dp[0] = 1;
        if (n >= 1) dp[1] = 1;

        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2];
        }
        return dp[n];
    }

    public static void main(String[] args) {
        System.out.println(ways(5)); // 8
    }
}
```

Complexity: Time O(n), space O(n); can be reduced to O(1).

9. Minimum Coin Change using tabulation

File: src/dp/tabulation/MinCoinsTabulation.java

Concept / state definition: dp[x] means minimum coins required for amount x. Initialize dp[0]=0 and all other states to INF. For each amount, try every usable coin.

```
package dp.tabulation;

import java.util.Arrays;

public class MinCoinsTabulation {
    static int minCoins(int[] coins, int amount) {
        int inf = amount + 1;
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, inf);
        dp[0] = 0;

        for (int x = 1; x <= amount; x++) {
            for (int coin : coins) {
                if (coin <= x && dp[x - coin] != inf) {
                    dp[x] = Math.min(dp[x], dp[x - coin] + 1);
                }
            }
        }
        return dp[amount] == inf ? -1 : dp[amount];
    }

    public static void main(String[] args) {
        System.out.println(minCoins(new int[]{1, 2, 5}, 11)); // 3
    }
}
```

Complexity: Time O(A·C), space O(A).

10. Side-by-side comparison

Point	Memoization	Tabulation
Direction	Top-down	Bottom-up
Main mechanism	Recursion + cache	Loops + table

Point	Memoization	Tabulation
Start point	Original requested state	Base/smallest states
State evaluation	On demand	Usually all states in chosen table
Stack	Uses call stack	No recursion stack
Risk	Stack overflow on deep recursion	Wrong loop order / table initialization
Best when	Only some states may be needed; recurrence is naturally recursive	Most states are needed; iterative performance/space matters
Debugging	Trace recursive calls and cache hits	Inspect table row-by-row
Space	Cache + recursion stack	DP table; often compressible
Typical conversion	Write recurrence first, add cache	Keep recurrence, derive dependency order

11. Dry run: Fibonacci(5)

Memoization call idea

```
fib(5)
 = fib(4) + fib(3)
 = (fib(3) + fib(2)) + (fib(2) + fib(1))
Repeated fib(3), fib(2) are computed once,
then fetched from memo.
memo[2]=1, memo[3]=2, memo[4]=3, memo[5]=5
```

Tabulation table

```
Index i :  0  1  2  3  4  5
dp[i]   :  0  1  1  2  3  5

dp[2] = dp[1] + dp[0] = 1
dp[3] = dp[2] + dp[1] = 2
dp[4] = dp[3] + dp[2] = 3
dp[5] = dp[4] + dp[3] = 5
```

12. How to convert memoization into tabulation

• Write the recursive state clearly, e.g. solve(i).

• Write all dependencies, e.g. solve(i-1), solve(i-2).

• Find base states and their values.

• Draw dependency arrows: i depends on smaller states.

• Choose a loop order that visits dependencies first.

• Replace recursive calls with dp array/table lookups.

• Check whether the full table is necessary; compress only after proving which old states are still needed.

Example conversion

```
Memoization:
int f(int i) {
    if (i <= 1) return i;
    if (memo[i] != -1) return memo[i];
    return memo[i] = f(i-1) + f(i-2);
}

Tabulation:
dp[0] = 0;
dp[1] = 1;
for (int i = 2; i <= n; i++)
    dp[i] = dp[i-1] + dp[i-2];
```

13. Space optimization

Space optimization is not a third DP technique. It is an optimization that can often be applied to tabulation when each state depends only on a small number of previous states.

14. Fibonacci with O(1) extra space

File: src/dp/optimized/FibonacciSpaceOptimized.java

Concept / state definition: Only dp[i-1] and dp[i-2] are needed. Keep those two values instead of the entire table.

```
package dp.optimized;

public class FibonacciSpaceOptimized {
    static long fib(int n) {
        if (n <= 1) return n;

        long prev2 = 0;
        long prev1 = 1;

        for (int i = 2; i <= n; i++) {
            long current = prev1 + prev2;
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }

    public static void main(String[] args) {
        System.out.println(fib(10)); // 55
    }
}
```

Complexity: Time O(n), extra space O(1).

15. Interview rules and common mistakes

• Do not confuse memoization with tabulation: memoization is top-down recursion + cache; tabulation is bottom-up iterative filling.

• Do not say tabulation is always faster in every practical case; actual performance depends on states visited, recursion overhead, memory layout, and implementation.

• Do not assume every DP can be space-optimized to O(1).

• Do not use 0 as an uncomputed marker if 0 is a valid result.

• Do not forget to initialize unreachable states to INF/false appropriately.

• Do not convert recursion to loops by guessing the order; derive it from dependencies.

• Do not confuse memoization with caching every possible input globally; cache the DP state relevant to the current problem.

• Be ready to discuss recursion depth, stack overflow, and memory trade-offs.

Which one should I use in an interview?

A reliable approach: derive the recurrence first. If the recursion is clear, implement memoization to validate the state and transition. Then, if iterative constraints or recursion depth make it preferable, convert to tabulation and optimize space where safe. You can also start directly with tabulation when the dependency order is obvious.

Practice checklist

• Fibonacci — memoization, tabulation, O(1) space.

• Climbing Stairs — memoization and tabulation.

• Min Cost Climbing Stairs — both approaches.

• House Robber I and II — recursive recurrence, memoization, tabulation, space optimization.

• Unique Paths and Unique Paths II — 2D memoization and 1D tabulation.

• 0/1 Knapsack — 2D memoization and 1D descending-capacity tabulation.

• Unbounded Knapsack / Coin Change — understand why capacity direction changes.

• LCS — 2D memoization, 2D tabulation, 1D compression.

• Edit Distance — memoization and tabulation.

• Longest Increasing Subsequence — O(n²) DP and O(n log n) tails method.

This guide fully implements representative Fibonacci, Climbing Stairs, and Minimum Coin Change versions in both styles, plus a space-optimized Fibonacci. The remaining practice checklist names problems to implement next; it does not claim each one has a full source file in this PDF.